# Predictive Parser and Lexical Analyzer Implementation for a Custom Programming Language

**Submitted By**


| Student Name | Student ID |
|---|---|
| MD. Hasibur Rahman | 241-15-806 |
| MD. Wahidur Rahman | 241-15-865 |
| MD. Rahul Hossain | 241-15-766 |
| MD. Arman Hossen Ripon | 241-15-883 |
| MD. Sabbir Hossine | 241-15-673 |


**LAB PROJECT REPORT**

This Report Presented in Partial Fulfillment of the Course: **CSE314**, Subject: **Compiler Design Lab** in the **Computer Science and Engineering Department**


**DAFFODIL INTERNATIONAL UNIVERSITY**
**Dhaka, Bangladesh**

**April 10, 2026**

# DECLARATION

We hereby declare that this lab project has been done by us under the supervision of **Partha Dip Sarkar**, **Lecturer**, Department of Computer Science and Engineering, Daffodil International University. We also declare that neither this project nor any part of this project has been submitted elsewhere as lab projects.

**Submitted To:**

_____

**Partha Dip Sarkar Lecturer**
Department of Computer Science and Engineering
Daffodil International University

**Submitted by**

| | |
|---|---|
| _____<br>Md Hasibur Rahman<br>ID: 241-15-806<br>Dept. of CSE, DIU | |
| _____<br>Md Wahidur Rahman<br>ID: 241-15-865<br>Dept. of CSE, DIU | _____<br>Md Arman Hossen Ripon<br>ID: 241-15-883<br>Dept. of CSE, DIU |
| _____<br>Md Rahul Hossain<br>ID: 241-15-766<br>Dept. of CSE, DIU | _____<br>Md Sabbir Hossine<br>ID: 241-15-673<br>Dept. of CSE, DIU |

# COURSE & PROGRAM OUTCOME

The following course have course outcomes as following:

Table 1: Course Outcome Statements

| CO's | Statements |
|---|---|
| CO1 | **Define and Relate** lexical tokens, grammar rules, and non-terminals needed for solving syntax validation problems. |
| CO2 | **Formulate** knowledge of automata theory and C programming to implement a functional predictive parser for problem solving. |
| CO3 | **Analyze** state transition matrices and predictive parsing tables to **Present** a specific formal language structure. |
| CO4 | **Develop** a compiler front-end for complex source code analysis while applying stack-based parsing concepts and evaluating effectiveness based on grammar standards. |

Table 2: Mapping of CO, PO, Blooms, KP and CEP

| CO | PO | Blooms | KP | CEP |
|---|---|---|---|---|
| CO1 | PO1 | C1, C2 | KP3 | EP1,EP3 |
| CO2 | PO2 | C2 | KP3 | EP1,EP3 |
| CO3 | PO3 | C4, A1 | KP3 | EP1,EP2 |
| CO4 | PO3 | C3, C6, A3, P3 | KP4 | EP1,EP3 |

The mapping justification of this table is provided in section **4.3.1**, **4.3.2** and **4.3.3**.

# Table of Contents

Declaration . . . . . i

Course & Program Outcome . . . . . ii

**1 Introduction . . . . . 1**

1.1 Introduction . . . . . **1**

1.2 Motivation . . . . . **1**

1.3 Objectives . . . . . **1**

1.4 Feasibility Study . . . . . **2**

1.5 Gap Analysis . . . . . **2**

1.6 Project Outcome . . . . . 2

**2 Proposed Methodology/Architecture . . . . . 3**

2.1 Requirement Analysis & Design Specification . . . . . **3**

2.1.1 Overview . . . . . **3**

2.1.2 Proposed Methodology/ System Design . . . . . **3**

2.1.3 UI Design . . . . . **4**

2.2 Overall Project Plan . . . . . **4**

**3 Implementation and Results . . . . . 5**

3.1 Implementation . . . . . **5**

3.2 Performance Analysis . . . . . **5**

3.3 Results and Discussion . . . . . **6**

**4 Engineering Standards and Mapping** . . . . . 7

4.1 Impact on Society, Environment and Sustainability . . . . . **7**

4.1.1 Impact on Life . . . . . **7**

4.1.2 Impact on Society & Environment . . . . . **7**

4.1.3 Ethical Aspects . . . . . **7**

4.1.4 Sustainability Plan . . . . . 8

4.2 Project Management and Team Work . . . . . **8**

4.3 Complex Engineering Problem . . . . . **8**

4.3.1 Mapping of Program Outcome . . . . . **9**

4.3.2 Complex Problem Solving . . . . . **9**

4.3.3 Engineering Standards and Mapping. . . . . 10

4.3.4 Complex Engineering Problem Knowledge profile . . . . . 10

4.3.5 Engineering Activities . . . . . 10

**5 Conclusion** . . . . . 11

5.1 Summary . . . . . **11**

5.2 Limitation . . . . . 11

5.3 Future Work . . . . . **11**

**References** . . . . . 12

# Chapter 1

# Introduction

This chapter provides a comprehensive overview of the design and objectives of the compiler project. It details the technical motivation behind the development of the lexical analyzer and predictive parser, while outlining the specific goals and expected outcomes of the system.

## 1.1 Introduction

This project involves the design and implementation of a front-end compiler system consisting of a Lexical Analyzer (Lexer) and a Syntax Analyzer (Parser). The system is designed to process an input stream, identify specific language tokens such as variables, functions, and loop labels, and validate the syntax against a predefined grammar using a predictive parsing table.

## 1.2 Motivation

The primary motivation for this project is to gain a practical understanding of how formal languages are processed using Finite State Machines (FSM) and non-recursive predictive parsing. By implementing these core components in C, we aim to solve the computational problem of efficiently tokenizing and validating code, which is an essential skill for software engineering and compiler design.

## 1.3 Objectives

1. **Token Recognition:** To implement a robust Lexer that distinguishes between keywords, variables, operators, and custom loop labels.
2. **Syntax Validation:** To develop a non-recursive Predictive Parser that utilizes a parsing table and a stack to verify the grammatical correctness of a program.
3. **Mandatory Entry Point:** To ensure that every validated program contains a `mainFn`, establishing a reliable entry point for execution.
4. **Error Reporting:** To provide clear error feedback when the input code contains invalid tokens or syntax mismatches.

## 1.4 Feasibility Study

This project builds upon established compiler design principles, such as the use of transition tables for lexical analysis and context-free grammars for syntax verification. Similar to industrial-standard compilers, our system uses a structured approach to process input files (`input.txt`) and manage token streams, making the implementation technically feasible within the C development environment.

## 1.5 Gap Analysis

While many basic compilers handle standard identifiers, this project addresses the need for a specialized analyzer that can handle custom constraints, such as specific alphanumeric patterns for variables and unique labeling requirements for loop structures. Our implementation fills the gap between simple expression evaluators and full-scale industrial compilers by providing a tailored front-end for a specific language grammar.

## 1.6 Project Outcome

The outcome of this work is a functional compiler front-end that successfully transforms raw text into a sequence of tokens and performs exhaustive syntax checking. The system accurately accepts or rejects input programs based on their adherence to the defined grammar, providing a solid foundation for future semantic analysis and code generation phases.

# Chapter 2

# Proposed Methodology

This chapter outlines the systematic approach used to design and implement the compiler's front-end components. It details the technical requirements and provides a architectural overview of how the Lexical Analyzer and Predictive Parser work together to process source code.

## 2.1 Requirement Analysis & Design Specification

The system is designed to handle a custom context-free grammar while operating within the memory constraints defined in the C implementation. Key requirements include support for a 300-symbol stack and a token stream of up to 300 lexemes.

### 2.1.1 Overview

The architecture follows a classic compiler front-end pipeline where raw characters from `input.txt` are converted into a discrete token stream. This stream is then fed into a non-recursive predictive parser that uses a stack-based algorithm to validate the program structure against the language rules.

### 2.1.2 Proposed Methodology

The project utilizes a deterministic approach to language processing:

1. **Lexical Analysis (FSM):** Employs an 8-state transition matrix (next_state[8][12]) to categorize input into 16 distinct terminal types, such as type, var, num, and loop.
2. **Predictive Parsing:** Implements a non-recursive LL(1) parsing strategy. It utilizes a 2D parsing table (TABLE[20][16]) to map 20 Non-Terminals to 36 specific production rules (RHS).
3. **Refinement Logic:** Includes custom sub-routines like (refine_id) and (valid_loop_label) to enforce specific naming conventions, such as variables following the (_a1b) pattern.

### 2.1.3 UI Design

While the project is a console-based compiler, the "User Interface" consists of structured terminal outputs:

1. **Lexer Table:** Displays a side-by-side mapping of each "Lexeme" to its identified "Token".
2. **Parser Trace**: Provides a real-time trace of the Stack, the current Lookahead symbol, and the specific Rule/Action being applied during validation.

## 2.2 Overall Project Plan

The development was partitioned into two primary phases:

1. **Lexer Construction:** Focusing on character-level state transitions and token identification.
2. **Parser Integration:** Focusing on grammar rule implementation, stack management, and mandatory logic checks, such as verifying the existence of the mainFn entry point.

# Chapter 3

# Implementation and Results

This chapter details the technical execution of the compiler project, from the initial environment setup to the final validation of the source code. It includes an analysis of the system's performance and a discussion of the outputs generated by both the lexical and syntax analysis phases

## 3.1 Implementation

The implementation was carried out using the C programming language to ensure low-level control over memory and state transitions.

1. **Development Environment:** The project was developed and tested using standard C compilers, focusing on modular functions for lexing and parsing.
2. **Data Structures: *Stack Management:** A character-based stack array stk[MAXSTACK][MAXSYM] was implemented to manage non-terminal expansions during predictive parsing.

    a) **State Transition Matrix:** An 8x12 integer matrix next_state[8][12] was utilized to drive the deterministic finite automaton for token recognition.

3. **Core Logic:** Sub-routines such as run_lexer and run_parser were integrated to handle sequential processing, where the lexer provides a token stream for the parser to validate against the defined TABLE[NNT][NTER].

## 3.2 Performance Analysis

The system's performance is characterized by its deterministic time complexity and efficient resource utilization.

1. **Lexical Efficiency:** The lexer processes input characters in a single pass, ensuring $O(n)$ time complexity relative to the number of characters in input.txt.
2. **Parser Determinism:** By using a predictive parsing table, the parser avoids the overhead of backtracking, allowing for rapid syntax validation.
3. **Memory Usage:** The program maintains a minimal memory footprint by utilizing fixed-size buffers (MAXSTACK, MAXTOK) to prevent dynamic allocation overhead during the critical parsing loop.

## 3.3 Results and Discussion

The project successfully achieves its goal of validating source code based on a specific context-free grammar.

1. **Lexical Output:** The lexer correctly identifies and labels diverse lexemes, including specialized loop labels (e.g., loop_abc01:) and variables following the mandatory _a1b pattern.

2. **Syntax Validation:** The parser provides a detailed trace of stack operations, matching terminals and expanding non-terminals until the entire input is consumed.

3. **Final Verification:** A critical success factor of the system is its ability to enforce the presence of the mainFn. If this mandatory function is missing, or if any syntax mismatch occurs, the system accurately outputs a **REJECTED** status; otherwise, it confirms the code is **ACCEPTED**.

# Chapter 4

# Engineering Standards and Mapping

This chapter outlines how the compiler project aligns with professional engineering standards and educational outcomes. It explores the societal and environmental implications of the software, provides a breakdown of project management, and justifies the attainment of specific Program Outcomes (POs) through technical implementation.

## 4.1 Impact on Society, Environment and Sustainability

The development of specialized compilers like this one plays a vital role in localizing software engineering capabilities and promoting efficient computational logic.

### 4.1.1 Impact on Life

By automating syntax validation and enforcing strict naming conventions for functions and loop labels, the project reduces the likelihood of logical errors in software, leading to more reliable applications for end-users.

### 4.1.2 Impact on Society & Environment

Efficient compilation (utilizing O(n) lexical passes and deterministic parsing) minimizes the CPU cycles required for code validation. This optimization contributes to reduced energy consumption during software development and deployment phases.

### 4.1.3 Ethical Aspects

The project adheres to academic integrity and ethical engineering practices by providing a transparent declaration of all team members' contributions and the software tools utilized during development.

### 4.1.4 Sustainability Plan

To ensure the project remains sustainable, the codebase is modularized. The use of a decoupled communication method (the output.txt file) allows for future expansion into semantic analysis or code generation without requiring a complete rewrite of the existing lexer and parser logic. This ensures that the front-end remains stable while the back-end evolves.

## 4.2 Project Management and Team Work

The project was managed through a collaborative effort among five team members, ensuring that tasks such as DFA design, predictive table construction, and C-code debugging were distributed effectively.

1. Resource Analysis: The primary resources utilized were the C standard library and local development environments.
2. Alternative Budgeting: Since this is an academic lab project, the "budget" primarily involved time-personnel hours; however, in an industry setting, the cost would scale based on the complexity of the grammar and the required level of error recovery sophistication.

## 4.3 Complex Engineering Problem

The development of a compiler front-end is classified as a complex engineering problem because it requires the integration of diverse high-level technical disciplines. It involves the translation of abstract mathematical theories—such as **Automata Theory** and **Formal Grammar**—into functional software that must operate with high precision and efficiency.

The complexity of this problem is rooted in three main areas:

1. **Mathematical Modeling:** Designing a Deterministic Finite Automaton (DFA) that correctly handles overlapping patterns (like distinguishing keywords from user-defined variables) requires rigorous logical mapping.
2. **Algorithmic Conflict Resolution:** Constructing an LL(1) predictive table requires resolving grammar ambiguities. Eliminating left-recursion and ensuring the grammar is non-ambiguous is a deep analytical task essential for linear-time parsing.
3. **System Integration:** The interdependence between the Lexical Analyzer and the Parser creates a "high-interaction" environment. A single character mismatch in the scanning phase can lead to a cascading failure in the syntactic validation phase, necessitating robust error-handling and synchronization logic.

By solving this problem, we demonstrate the ability to apply engineering knowledge to build a structured tool capable of interpreting and validating custom programming syntax.

### 4.3.1 Mapping of Program Outcome

In this project, the development of a two-stage compiler front-end demonstrates a systematic approach to solving engineering problems through mathematical modeling (DFA) and algorithmic implementation (LL(1) Parsing).

Table 4.1: Justification of Program Outcomes

| PO's | Justification |
|---|---|
| PO1 | **Engineering Knowledge:** Applied fundamental principles of mathematics and computer science by using Deterministic Finite Automata (DFA) for tokenization and Context-Free Grammar (CFG) for syntax analysis. |
| PO2 | **Problem Analysis:** Analyzed the complexity of source code strings to identify valid lexemes and used predictive parsing tables to detect syntactic errors in structured input. |
| PO3 | **Design/Development of Solutions:** Designed a modular solution consisting of a Lexical Analyzer and a Top-Down Parser, capable of transforming raw text into a structured sequence of grammar-validated tokens. |

### 4.3.2 Complex Problem Solving

The project addresses complex problem-solving through several engineering layers:

- **Engineering Standards and Mapping:** The project adheres to standard compiler design phases. The Lexical Analyzer acts as a scanner, while the Parser acts as a recognizer, mirroring industry-standard front-end architectures like those used in GCC or Clang.

- **Complex Engineering Problem Knowledge:** The challenge involved mapping a theoretical Grammar to a numerical Predictive Parsing Table. This required solving the "First and Follow" set logic to ensure the grammar was non-ambiguous and LL(1) compatible.

- **Rationale of Profile:** The solution is modularized. By separating the Lexer logic from the Parser logic, the system can be updated to support new language features (like additional operators or loop structures) by simply modifying the DFA transition table or the LL(1) production rules, without re-engineering the entire codebase.

### 4.3.3 Engineering Standards and Mapping

This project adheres to formal engineering standards in the field of Computer Science and Software Engineering. By following these standards, the project ensures reliability, maintainability, and compatibility with established computational theories.

### 4.3.4 Complex Engineering Problem Knowledge profile and rational thereof

The design demonstrates the application of complex engineering knowledge by:

1. Converting abstract mathematical sets (First and Follow sets) into a numerical **Predictive Parsing Table**.
2. Handling complex lexemes (like loop_main01:) through prioritized string pattern matching, ensuring that the system can distinguish between general identifiers and specialized control structures.

Table 4.2: Mapping with complex problem solving.

| EP1 Dept of Knowledge | EP2 Range of Conflicting Requirements | EP3 Depth of Analysis | EP4 Familiarity of Issues | EP5 Extent of Applicable Codes | EP6 Extent of Stak e- holder Involvement | EP7 Interdependence |
|---|---|---|---|---|---|---|
| √ | √ | | | | | |

### 4.3.5 Engineering Activities

In this section, provide a mapping with engineering activities. For each mapping add subsections to put rationale (Use Table 4.3).

Table 4.3: Mapping with complex engineering activities.

| EA1 Range of re- sources | EA2 Level of Interaction | EA3 Innovation | EA4 Consequences for society and environment | EA5 Familiarity |
|---|---|---|---|---|
| √ | √ | | | |

# Chapter 5

# Conclusion

This chapter provides a final overview of the project's achievements, acknowledges the current technical constraints, and proposes potential avenues for expanding the compiler's capabilities.

## 5.1 Summary

The project successfully implemented a functional compiler front-end consisting of a **Deterministic Finite Automata (DFA) based Lexical Analyzer** and a **Predictive LL(1) Parser**. The system effectively processes raw source code from `input.txt`, categorizes lexemes into distinct tokens (such as `VARIABLE`, `LOOP`, and `OPERATOR`), and validates their syntactic structure against a predefined Context-Free Grammar. The use of an intermediate `output.txt` file demonstrated a robust modular design, ensuring that the parser correctly recognizes valid code strings with an **"ACCEPTED"** status.

## 5.2 Limitation

While the project achieves its core objectives, it has certain limitations:

1. **Error Recovery:** The current LL(1) parser is designed to stop execution upon encountering the first syntax error, lacking advanced "panic-mode" or phrase-level error recovery.
2. **Grammar Scope:** The parser is currently restricted to a specific subset of expressions. It does not yet support complex nested logic, such as multi-level pointer arithmetic or complex data structures (structs/unions).
3. **Lexical Rigidity:** The DFA requires manual state updates to include new symbols, making it less flexible than table-driven scanners or lexer generators like Flex.

## 5.3 Future Work

Future iterations of this project could focus on:

1. Semantic Analysis: Implementing a Symbol Table and Type Checker to ensure that variables are declared before use and that operations are type-compatible.
2. Code Generation: Expanding the pipeline to generate Intermediate Representation (IR) code or assembly-level instructions for a virtual machine.
3. Enhanced Lexer Logic: Integrating a more dynamic lexical scanner capable of handling multi-line comments and escape sequences more efficiently.

# References

[1] Jon Kleinberg and Eva Tardos. *Algorithm Design*. Pearson Education India, 2006.

[2] Alfred V. Aho, Monica S. Lam, Ravi Sethi, and Jeffrey D. Ullman. *Compilers: Principles, Techniques, and Tools* (2nd Edition). Pearson, 2006.

[3] Dick Grune and Ceriel J.H. Jacobs. *Parsing Techniques: A Practical Guide*. Springer Science & Business Media, 2007.